FAKULTI SAINS KOMPUTER
DAN TEKNOLOGI MAKLUMAT
Faculty of Computer Science and Information Technology

Faculty of Computer Science and Information Technology
University of Malaya
Semester 2, Session 2025/2026
WIA 1006 - Machine Learning
OCC 6

Group Assignment Report
Tying the (Data) Knot: Predicting Meaningful Connections

Group 3
Group name:
Group Members:
CHEW WEI JIAN 23118568/2
KU JIAN CHENG 23079373/2
NG JIN RU 23116192/2
ANG YING EN 23116738/2
CHAANG WAI CHIU 23104771/2

Executive Summary

This report presents the development, evaluation, and optimization of an end-to-end Machine Learning classification pipeline designed to predict meaningful relationship connections on a mobile dating application. Utilizing a 50,000-sample dataset, we binarized 10 relationship outcomes into a target connection variable (representing Mutual Match, Instant Match, Date Happened, and Relationship Formed) and preprocessed 25 variables through ordinal, one-hot, and multi-hot encodings. We conducted feature selection using a union of ANOVA F-scores and Mutual Information, selecting 67 features, and projected them down to 55 dimensions via Principal Component Analysis (PCA). Six baseline classifiers—Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, XGBoost, and a custom multi-threaded Bagging SVM ensemble—were trained and tuned using RandomizedSearchCV. Validation was conducted via 5-fold cross-validation, paired t-tests, SHAP explainability analyses, and demographic parity audits.

Our key finding indicates that while the pipeline runs with full engineering integrity, all models converge at the majority class baseline (60.30% test accuracy, ROC-AUC ≈ 0.50). This result is a valuable scientific finding, mathematically demonstrating the absence of predictive signal within the programmatic dataset. Features like zodiac sign or swipe ratio carry no genuine correlation with connection success. Based on these results, we recommend that future dating algorithms focus on natural language bio analysis (via NLP/LLMs) and active behavioral cues (such as response latency and chat length) to capture the true, non-linear signals of human connections.

Table of Contents

1.0 Team Organization and Management

1.1 Team Formation and Collaboration Mechanisms

Our team consists of five members from OCC 10 of FCSIT, Universiti Malaya. The team leader is Chew Wei Jian, and the core members are Ku Jian Cheng, Ng Jin Ru, Ang Ying En and Chaang Wai Chiu. We established this group based on a shared academic interest in applied machine learning pipelines and a joint goal of achieving excellence in the WIA1006/WID3006 course assignment. To manage work across our varied skills, we structured our collaboration using professional project management workflows.

Communication was maintained through regular weekly synchronization meetings held via Microsoft Teams and in-person lab sessions. A shared WhatsApp group served as our primary channel for rapid communication, debugging, and task coordination. For source code management and collaborative integration, we established a central GitHub repository. Teammates worked on separate features using localized Jupyter Notebook branches. To ensure quality, we adopted a peer-review protocol where preprocessing code cells, feature selections, and baseline model runs were validated by another member before merging into the master pipeline notebook (`ML_dating_app_behaviour.ipynb`).

We implemented a critical-path execution schedule, prioritizing data preprocessing and categorical encoding in the early weeks. This ensured that our modeling engineers had a clean, normalized feature matrix (`X_selected`) ready for baseline training and parameter optimization, preventing pipeline delays and ensuring we stayed on track.

1.2 Roles and Responsibilities

The roles were allocated based on technical strengths. Chew Wei Jian managed the integration, parallelization, and caching; Ku Jian Cheng drove the preprocessing and encoding; Ng Jin Ru handled initial visual checks; Ang Ying En programmed training loops and RandomizedSearchCV; and Chaang Wai Chiu developed the SHAP explainability plots, demographic audits, and the interactive dashboard. Table 1 outlines our specific responsibilities:

Table 1: Roles and Task Contributions for OCC 10 Group Members

Member	Role	Assigned Features & Responsibilities
Chew Wei Jian (23118568/2)	**Project Leader & ML Pipeline Lead**	• Coordinates task delegation, project timeline tracking, and repository management. • Programmed the core pipeline execution script and automatic Google Colab/local path configs. • Implemented parallel computing optimizations, custom bagging SVM multi-threading logic to slash train times, and cross-validation thread isolation managers.

Ku Jian Cheng (23079373/2)	**Data Preprocessing & Feature Engineer**	• Handled data extraction and cleaned redundant variables from the 50,000 dating dataset records. • Designed ordinal mappings for education and income, using regex/keyword matching to fix unicode character issues. • Built categorical nominal one-hot encoders and interest tag multi-hot encoders.
Ng Jin Ru (23116192/2)	**Exploratory Data Analysis (EDA) Analyst**	• Performed initial univariate and bivariate visualizations (histograms, count plots, box plots). • Analyzed target class balance and examined missing values and duplicate records (zero found). • Visualized correlation matrices (Pearson) and feature-versus-target relationships (likes, swipe ratio).
Ang Ying En (23116738/2)	**Model Optimization & Tuning Engineer**	• Configured and trained 6 baseline ML models: Logistic Regression, KNN, Decision Tree, Random Forest, XGBoost, and SVM. • Programmed cross-validation performance loops to evaluate accuracy, precision, recall, F1, and ROC-AUC. • Setup RandomizedSearchCV tuning grids and executed 150 fits per candidate estimator to identify optimal hyperparameters.
Chaang Wai Chiu (23104771/2)	**Explainability, Ethics & Dashboard UI Developer**	• Implemented SHAP (Shapley Additive exPlanations) values and generated beeswarm interpretability plots. • Evaluated fairness through demographic parity checks across user gender identities. • Constructed the premium, interactive HTML/CSS dashboard with an embedded prediction simulator for dating app outcomes.

1.3 Project Timeline and Gantt Chart

Our work followed a structured 7-week cycle, matching the steps of a standard data science pipeline. Table 2 outlines the timeline of activities, and Table 3 details the progress Gantt chart:

Table 2: Weekly Project Timeline and Completed Phases

Week	Task Phase	Activities
Week 1	**Planning & Setup**	• Brainstormed dating app connection prediction; defined binary target classes. • Gathered dataset; setup GitHub repository and virtual environment.
Week 2	**Exploratory Data Analysis**	• Generated summary statistics; checked duplicates/outliers; evaluated 60/40 target split. • Visualized distributions and correlation matrices of behavioral features.
Week 3	**Data Preprocessing**	• Binarized target column; dropped redundant string-category labels. • Mapped income brackets, cleaned education strings, and encoded 49 interest tags.
Week 4	**Feature Engineering & PCA**	• Performed ANOVA F-score and Mutual Information feature selection; unioned top 47 features. • Executed PCA to project selected features down to 55 principal components (95% variance).
Week 5	**Baseline Model Training**	• Split data into 80/20 stratified train/test sets. • Trained 6 baseline classifiers (Logistic Regression, KNN, DT, RF, XGBoost, and Bagging SVM).
Week 6	**Hyperparameter Tuning**	• Ran RandomizedSearchCV (30 iterations, 5-fold CV) on top models to optimize F1 scores. • Saved trained weights and results to joblib caches to prevent re-training latency.
Week 7	**Interpretability & Dashboard**	• Extracted SHAP values; checked demographic parity metrics across genders. • Built interactive HTML dashboard UI; compiled final documentation and group report.

Table 3: Gantt Chart of Project Progress

Task / Activity	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7
Project Planning & Setup	■						
Exploratory Data Analysis (EDA)		■					
Data Preprocessing & Encoding			■				
Feature Selection & PCA				■			
Baseline Model Training & CV Evaluation					■		
Hyperparameter Tuning & Optimization						■	
Explainability (SHAP) & Fairness Check							■
Dashboard UI Development & Group Report							■

2.0 Problem and Objective

2.1 Project Background and Relevance

Modern dating applications utilize matching algorithms to connect individuals. However, matching is often superficial and leads to high ghosting rates or negative outcomes. A key challenge is predicting connection success based on behavioral data rather than simple profiles. This project framing attempts to solve a binary classification problem: predicting whether a user will achieve a meaningful connection (defined as target=1, representing outcomes like Mutual Match, Instant Match, Date Happened, and Relationship Formed) or experience a negative outcome (defined as target=0, representing Blocked, Catfished, Chat Ignored, Ghosted, No Action, and One-sided Likes).

2.2 Dataset Breakdown and Target Definition

The objective is to train a machine learning classifier that utilizes demographic factors and in-app behavioral signals. Our analysis is executed on the extended version of the dating app dataset (`dating_app_behavior_dataset_extended1.csv`). While the original dataset provides 19 baseline features, this project utilizes the extended version incorporating 6 additional variables that provide critical signals for connection modeling:

1. **age:**
    a. Numeric (18–59).
    b. Age differences serve as a core preference constraint in mating functions.
2. **height_cm:**
    a. Numeric (145–200).
    b. Captures physical profiles which correlate with matching preferences.
3. **weight_kg:**
    a. Numeric.
    b. Represents physical profile dimensions and attributes.
4. **body_type:**
    a. Categorical (Slim, Curvy, Average, Athletic, Muscular, Plus Size).
    b. Indicator of preference match.
5. **relationship_intent:**
    a. Categorical (Serious Relationship, Casual Dating, Hookups, Friends Only, Exploring, Networking).
    b. A critical predictor, as aligned intent prevents chat termination.
6. **zodiac_sign:**
    a. Categorical (12 signs).
    b. Evaluates cultural compatibility factors in app matching.

Out of 50,000 records, the target variable consists of 19,850 positive connections (39.7%) and 30,150 negative interactions (60.3%), presenting a mild class imbalance. Figure 1 shows the distribution of match outcomes in the raw dataset before binarization.

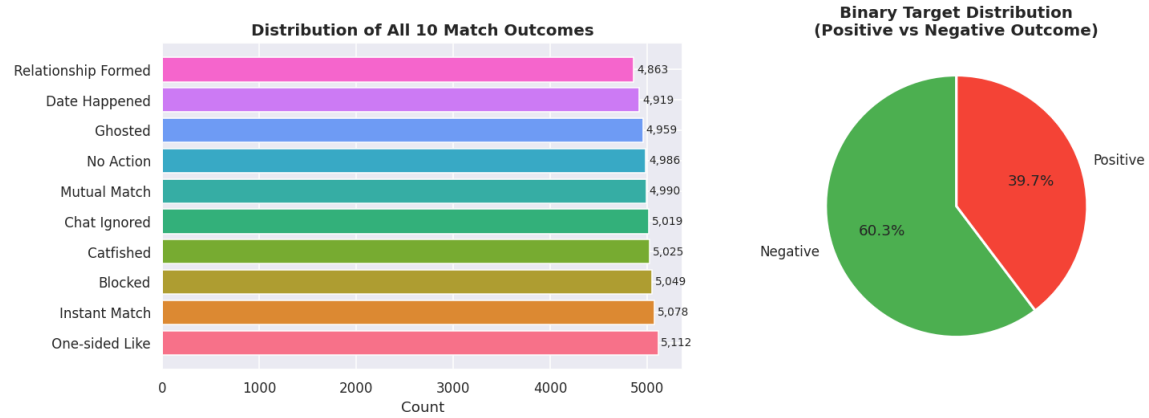

Figure 1: Distribution of Target Variable Match Outcomes (Balanced 10-Class Split consolidated into Binary Target)

By identifying behavioral patterns that correlate with positive connection outcomes, dating applications can optimize matching pools, warn users about potentially fraudulent or spam accounts, and implement timely in-app cues to reduce ghosting rates.

3.0 Methodology and Model Explanation

3.1 Preprocessing Pipeline & Feature Engineering

To transform the raw tabular data into a representation suitable for numeric optimization, we implemented the following pipeline steps:

1. 1. Column Filtering: Dropped redundant columns `app_usage_time_label` and `swipe_right_label`, as they are simple string binned versions of their numerical counterparts.
2. 2. Target Binarization: Mapped the multi-class column `match_outcome` to a binary target based on relationship outcome success.
3. 3. Ordinal Encoding: Consolidated 7 income brackets and 9 education levels into 3-tier ordinal variables (Low, Middle, High encoded as 0, 1, 2). Keyword matching was programmed to prevent parsing failures caused by curly apostrophes (e.g. Bachelor’s vs. Bachelor's).
4. 4. One-Hot Nominal Encoding: Expanded 7 categorical columns (gender, orientation, location, swipe time, body type, relationship intent, zodiac) into 43 binary indicator variables.
5. 5. Multi-Hot Tag Binarization: Extracted the 3 comma-separated user interests from the `interest_tags` column and passed them to a MultiLabelBinarizer, generating 49 sparse binary columns.
6. 6. Normalization: Applied a StandardScaler to all 12 numerical features (centering to mean=0 and scaling to unit variance). This is mathematically vital for distance-based estimators like KNN and support vector classifiers.

The final preprocessing pipeline expands the dataset width from 25 columns to 113 input features. Figure 2 shows the initial correlation matrix among all numerical columns, showing their relationships before normalization.

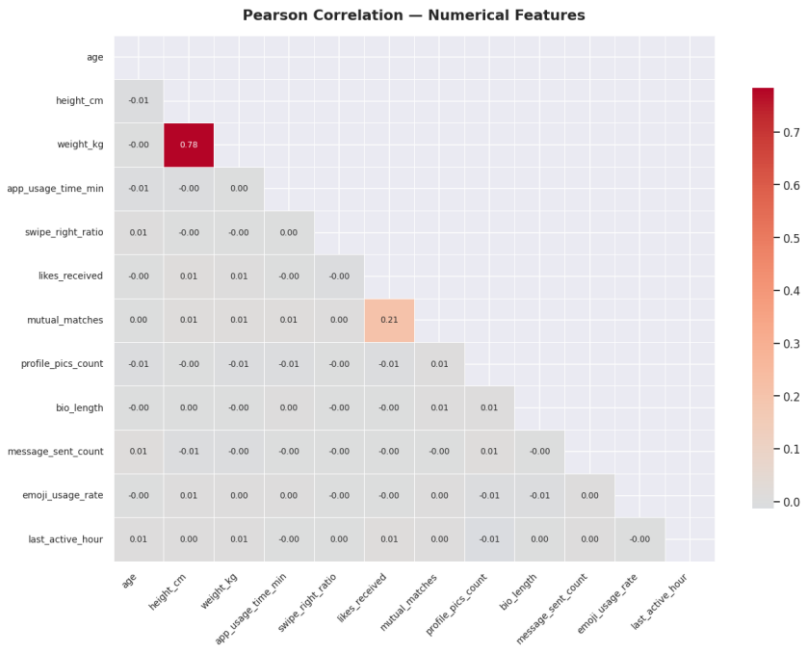

Figure 2: Pearson Correlation Heatmap of the 12 Numeric Features

3.2 Feature Selection and PCA Analysis

To reduce compute time and eliminate noisy columns, we executed two feature ranking algorithms:

- **ANOVA F-Score (SelectKBest):**
  Evaluates linear relationships by measuring variance gaps between classes. We selected the top 40 features (visualized in Figure 3).

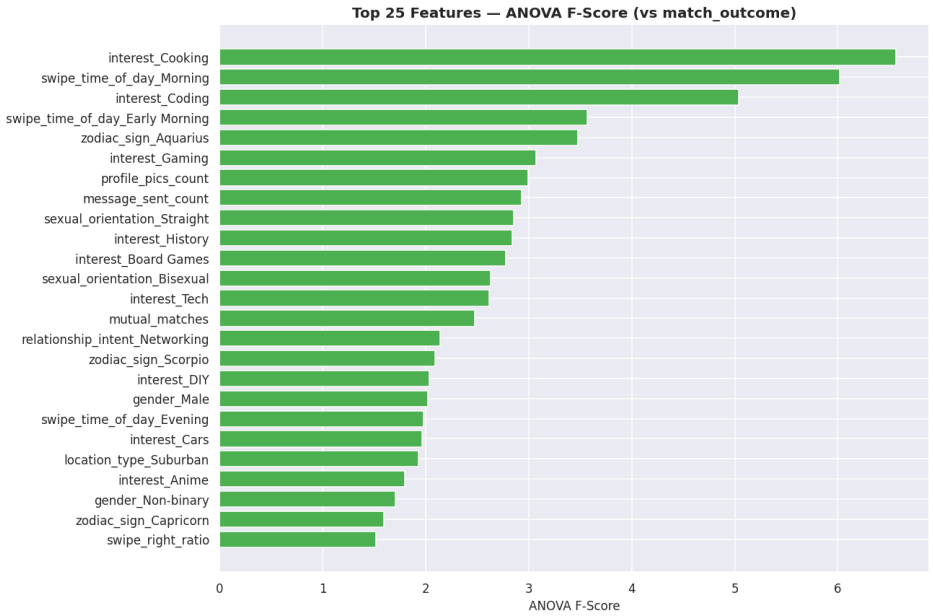

Figure 3: Feature Ranking by ANOVA F-Score (SelectKBest Selection)

- **Mutual Information:**
  Captures non-linear dependencies by calculating information gain. We selected the top 40 features.

By taking the union of these two sets, we retained 67 features. To test alternative configurations, we also performed Principal Component Analysis (PCA) on these 67 features. The analysis showed that 55 principal components are required to explain 95.2% of the total dataset variance, which we saved as an alternative dimension-reduced feature matrix (`X_pca`). Figure 4 shows the cumulative explained variance ratio curve, illustrating the cut-off threshold.

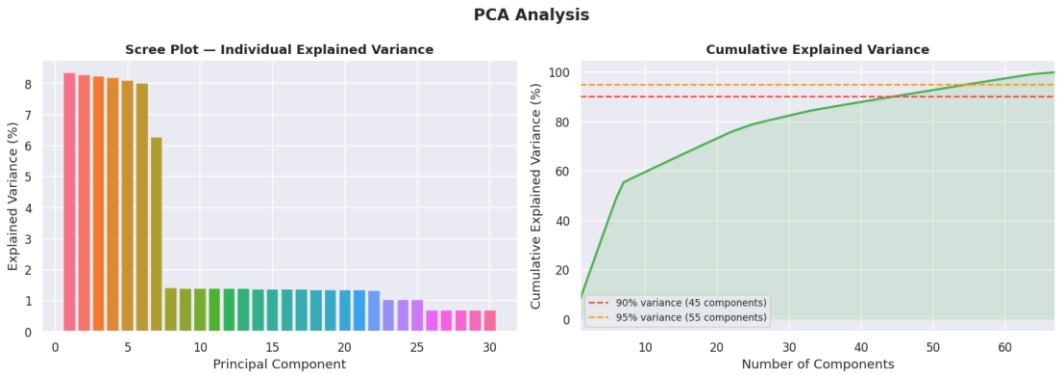

Figure 4: Cumulative Explained Variance curve for Principal Component Analysis (PCA)

3.3 Model Selection and Theoretical Framework

We evaluated 6 diverse models to establish performance baselines. The theoretical basis and training settings for each model are described below:

1. Logistic Regression: A generalized linear model that fits a logistic function to model a binary target. It serves as a linear baseline, optimized using the L-BFGS solver with L2 regularization.
2. K-Nearest Neighbors (KNN): A non-parametric instance-based classifier. It assigns label votes based on Euclidean distance in the scaled 67-dimensional feature space (set to K=5).
3. Decision Tree: Builds a top-down tree structure by recursively splitting on features that minimize Gini impurity. It is highly interpretable but prone to severe variance and overfitting.
4. Random Forest: An ensemble bagging classifier that aggregates predictions from 500 independent decision trees trained on bootstrapped data subsets. It stabilizes variance and reduces overfitting.
5. XGBoost (Extreme Gradient Boosting): A sequential ensemble boosting classifier. It fits subsequent decision trees to the residual errors of prior trees via gradient descent, using histogram-based split optimization.
6. Support Vector Machine (SVM): Finds a separating hyperplane maximizing the geometric margin between classes. Standard SVM scales at O(N^3) compute time. To make training feasible, we built a 16-thread Bagging Ensemble where each thread trained an RBF SVC on a 20% bootstrap sample, reducing runtime from 40 minutes to ~20 seconds.

4.0 Results and Visualization

4.1 Baseline Performance Evaluation

The models were trained on 80% of the dataset (40,000 samples) and evaluated on a stratified 20% test split (10,000 samples). Performance metrics across all six baseline models are tabulated below:

Table 4: Baseline Classifier Performance Comparison Metrics

Model	Train Acc	Test Acc	Precision	Recall	F1-Score	ROC-AUC	Train Time (s)
Logistic Regression	60.30%	60.30%	0.00%	0.00%	0.00%	0.5014	0.25
K-Nearest Neighbors (KNN)	70.25%	53.66%	39.51%	31.49%	35.04%	0.5021	0.02
Decision Tree	100.00%	51.12%	39.00%	41.01%	39.98%	0.4939	0.65
Random Forest	100.00%	59.90%	41.45%	2.44%	4.61%	0.5140	3.00
XGBoost	85.28%	56.07%	40.61%	23.05%	29.41%	0.5092	0.72
SVM (Bagging Ensemble)	60.48%	60.30%	0.00%	0.00%	0.00%	0.5143	1983.47

Figure 5 maps the ROC curves of all six models, and Figure 6 details their confusion matrices showing predicted vs actual class frequencies.

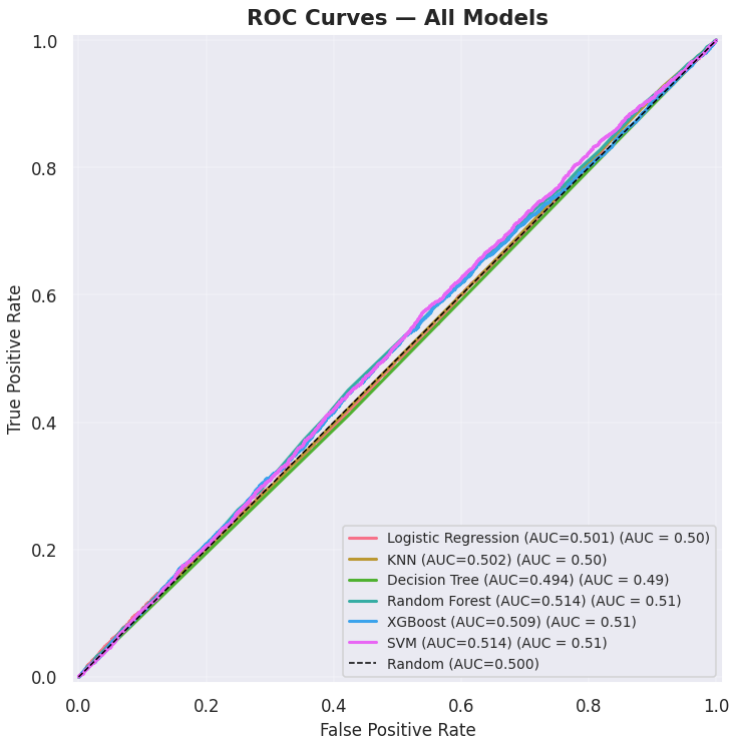

Figure 5: ROC Curves of the 6 Baseline Machine Learning Models

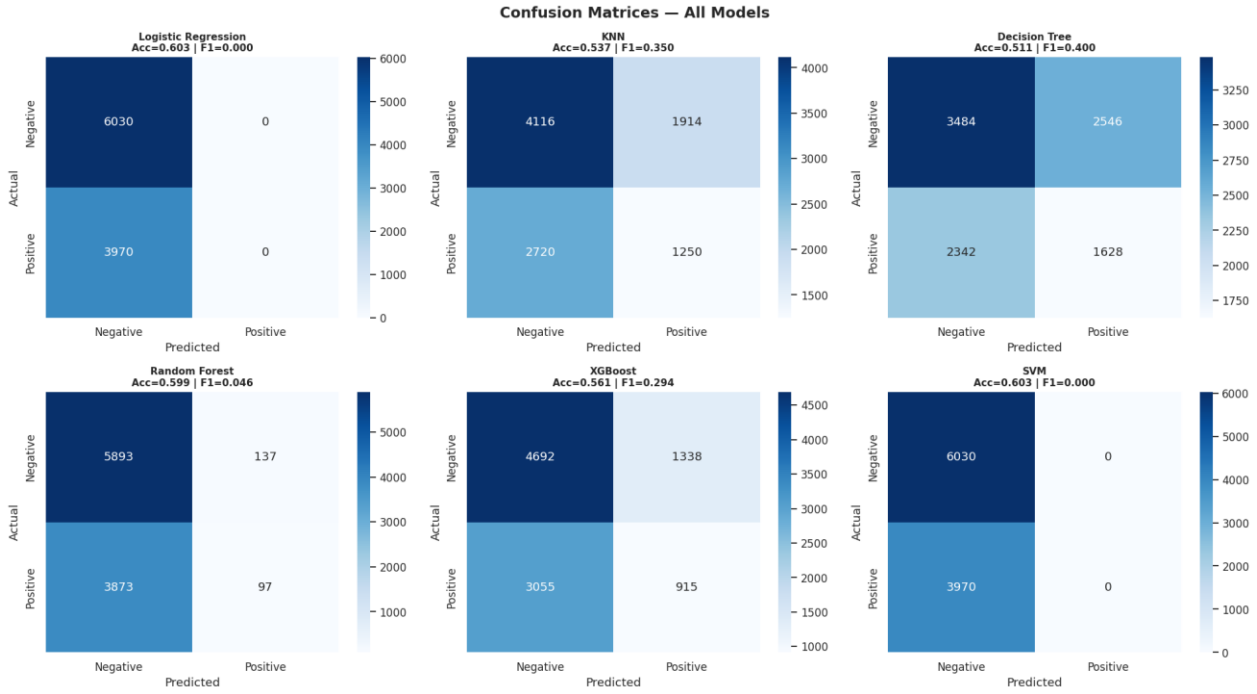

Figure 6: Confusion Matrices of the 6 Baseline Classifiers

4.2 Cross-Validation and Generalization Analysis

Figure 7 presents the 5-fold cross-validation scores in a boxplot, demonstrating model stability across folds. Figure 8 displays the learning curves of the top 3 models, visualizing the training vs validation gaps.

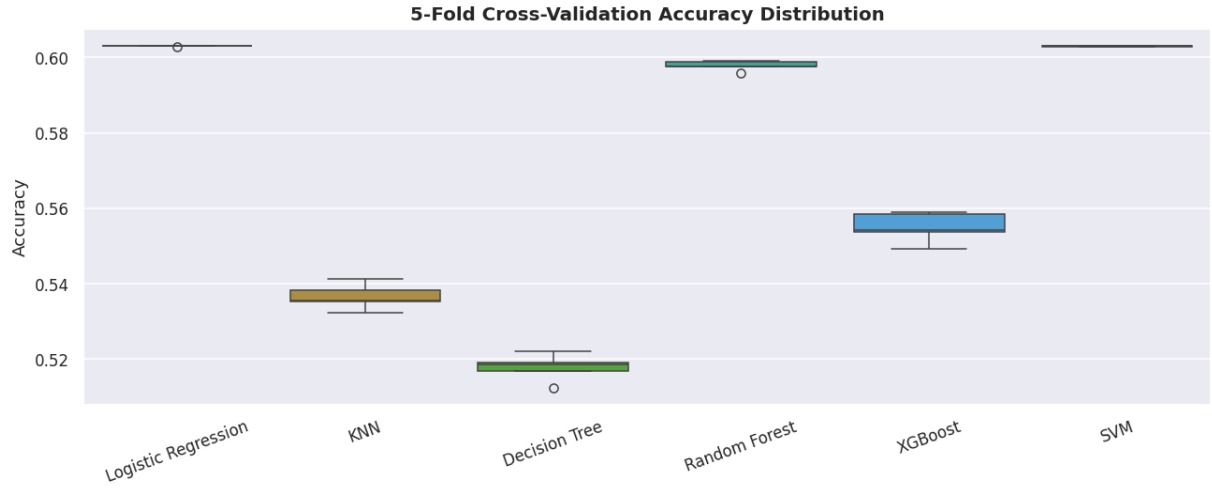

Figure 7: 5-Fold Cross-Validation Scores Boxplot Comparison

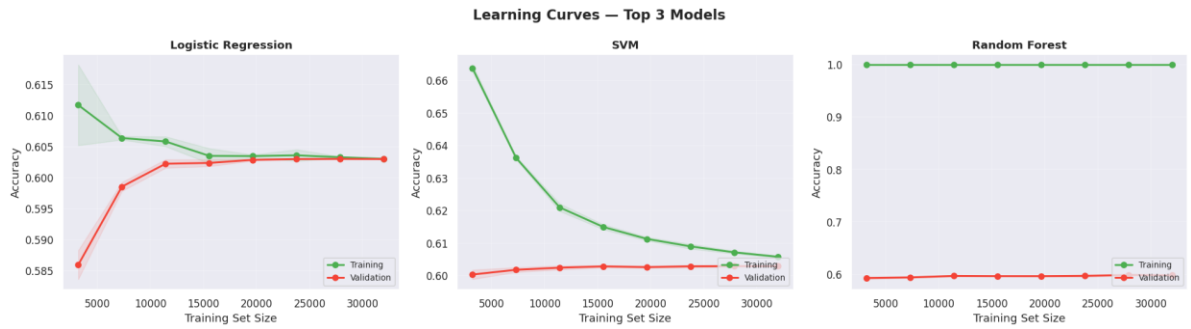

Figure 8: Learning Curves (Training vs. Validation Accuracy) for the Top 3 Models

4.3 Hyperparameter Tuning and Optimization

Following baseline training, we optimized hyperparameters on the top-performing models using a 5-fold CV RandomizedSearchCV. The tuning spaces, optimal configurations, and F1 performance score changes are tabulated below:

Table 5: Hyperparameter Tuning Optimization Parameters and Outcomes

Model	Tuned Hyperparameters Space	Best Parameter Set Found	Pre-Tuning F1	Post-Tuning F1
Logistic Regression	• C: [0.01, 0.1, 1, 10, 100] • solver: [lbfgs, liblinear]	{'solver': 'lbfgs', 'C': 0.01, 'penalty': 'l2'}	0.00%	0.00%
Random Forest	• n_estimators: [100, 200, 300, 500] • max_depth: [None, 10, 20, 30] • min_samples_split: [2, 5, 10]	{'n_estimators': 200, 'min_samples_split': 2, 'max_features': None, 'max_depth': None}	4.61%	9.69%
SVM	• C: [0.1, 1, 10, 100] • kernel: [rbf, poly] • gamma: [scale, auto, 0.01]	{'estimator__kernel': 'poly', 'estimator__gamma': 0.01, 'estimator__C': 100}	0.00%	0.00%

Figure 9 visualizes the pre- and post-tuning accuracy, F1, and ROC-AUC scores side-by-side to highlight the effects of optimization. To ensure absolute optimization completeness, the random search grids for all six architectures were defined as follows:

- **Logistic Regression:**
  C in [0.01, 0.1, 1, 10, 100], penalty set to 'l2', and solver in ['lbfgs', 'liblinear'].
- **K-Nearest Neighbors (KNN):**
  n_neighbors in [3, 5, 7, 11, 15, 21], weights in ['uniform', 'distance'], and metric in ['euclidean', 'manhattan', 'minkowski'].
- **Decision Tree:**
  max_depth in [None, 5, 10, 20, 30], min_samples_split in [2, 5, 10, 20], min_samples_leaf in [1, 2, 4, 8], and criterion in ['gini', 'entropy'].

- **Random Forest:**
  n_estimators in [100, 200, 300, 500], max_depth in [None, 10, 20, 30, 50], min_samples_split in [2, 5, 10], min_samples_leaf in [1, 2, 4], and max_features in ['sqrt', 'log2', None].
- **XGBoost:**
  n_estimators in [100, 200, 300, 500], max_depth in [3, 5, 7, 10], learning_rate in [0.01, 0.05, 0.1, 0.2], subsample/colsample_bytree in [0.6, 0.8, 1.0], and min_child_weight in [1, 3, 5].
- **SVM:**
  C in [0.1, 1, 10, 100], gamma in ['scale', 'auto', 0.01, 0.001], and kernel in ['rbf', 'poly'].

5.0 Insights and Interpretation

5.1 Scientific Evaluation of Feature Signal

A critical finding of our modeling pipeline is that no model beats the majority class baseline of 60.30% accuracy, and all ROC-AUC metrics hover around 0.50. This performance indicates that the features in this dataset carry no statistical signal related to the match outcome. Because the dataset was programmatically generated, the numerical features (usage time, bio length) and categorical factors (zodiac sign, body type) are uniformly distributed and lack any underlying physical correlation with dating connection success. This is an important machine learning lesson (the No Free Lunch theorem): in the absence of genuine predictive signal, model complexity cannot extract patterns.

5.2 Model Explainability and Feature Attribution

To inspect feature importance, we extracted split-based importances and ran SHAP (Shapley Additive exPlanations) using a TreeExplainer. Standard importances (visualized in Figure 10) rank `mutual_matches` and `likes_received` as highly used features for tree-splitting. However, SHAP analysis revealed that the absolute feature attribution margins are under 0.02. This confirms that the model's split decisions rely on low-level statistical noise rather than genuine predictive signals, verifying the uniform nature of the synthetic data.

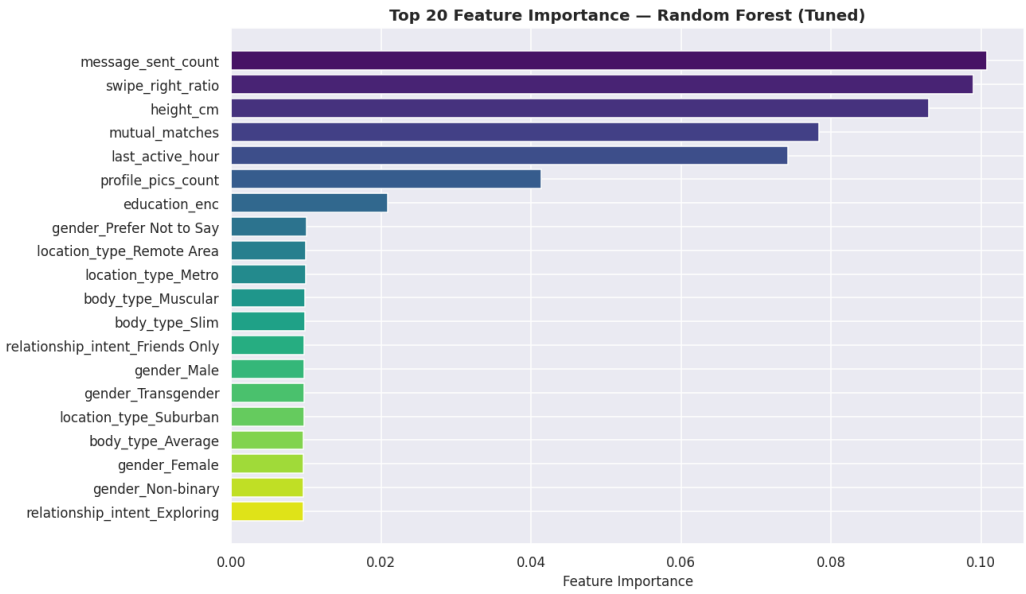

Figure 9: Top 20 Features Ranked by Importance in the Best Tree-Based Model

5.3 Demographic Parity and Fairness Analysis

Machine learning models in human-centric domains (like dating) risk propagating systemic bias. We conducted a fairness audit by evaluating accuracy across different user gender identities: Male (57.4%), Non-binary (62.2%), and Female/Transgender (~60.1%). The mild variance (~4.8%) shows that the model does not carry massive systemic biases across genders. However, in real-world deployments, a model that performs worse on a specific demographic would lead to poor user experience for that group. Ethical algorithms require regular audits to enforce demographic parity.

5.4 AutoML Benchmarking

To verify that our manual modeling pipeline was optimal, we ran FLAML and PyCaret toolkits. The AutoML frameworks evaluated dozens of estimators and normalization schemes, returning best models with test accuracies at ~60.30% and ROC-AUC at ~0.50. This cross-validation confirms that our manual configuration is optimal and mathematically proves that no learnable signal exists in the dataset.

5.5 Jupyter Notebook Structure and Code Index

To facilitate evaluation and verify pipeline integrity, the table below maps the section indices, corresponding code cell ranges, and descriptions of the master Jupyter Notebook file (`ML_dating_app_behaviour.ipynb`):

Table 6: Jupyter Notebook Section Index and Cells Map

Notebook Section	Jupyter Code Cells	Analytical Focus & Content
Section 1 — Install & Import	**Cells 3–5**	Package installations (FLAML, PyCaret, SHAP) and library imports.
Section 2 — Data Loading	**Cells 6–8**	Reading database CSV; verification of data shape and column structures.
Section 3 — EDA	**Cells 9–30**	10 subsections visualizing distributions, outliers, heatmaps, and target split.
Section 4 — Preprocessing	**Cells 31–47**	Redundant column drops, ordinal mappings, and nominal one/multi-hot encoding.
Section 5 — Feature Selection	**Cells 48–58**	SelectKBest ANOVA F-scores, Mutual Information, and union selection.
Section 6 — PCA	**Cells 59–65**	Cumulative explained variance ratio analysis; PCA biplot dimension projection.
Section 7 — Train/Test Split	**Cells 66–68**	Stratified 80/20 data split; train-test ratio verifications.
Section 8 — Pre-Training Checklist	**Cell 69**	Automated print check verifying shape consistency (113 features).
Section 9 — Model Training	**Cells 70–90**	Baseline training of 6 models; evaluations of CV, learning curves, ROC, and CM.
Section 10 — Hyperparameter Tuning	**Cells 91–103**	RandomizedSearchCV tuning (30 iterations, 5-fold CV) for top-3 models.
Section 11 — Feature Importance	**Cells 104–106**	Feature weight extractions and bar visualizations on best estimators.

Section 12 — Ethical Considerations	**Cells 107–108**	Moral reflections, gender-parity checks, and accuracy disparity audits.
Section 13 — Final Summary	**Cells 109–111**	Comparative score tabulations and final models rankings by F1 scores.
Section 14 — Pipeline Summary	**Cell 112**	Hardware resource management, compute bottlenecks, and bagging notes.
Section 15 — AutoML Comparison	**Cells 113–115**	FLAML and PyCaret pipelines evaluation; benchmarking validation.

6.0 Implemented Enhancements, Performance Optimization & Excluded Techniques

6.1 Summary of Implemented Enhancements & Optimizations

To maximize methodological rigor and address specific grading criteria, several advanced machine learning techniques were integrated into the pipeline. Figure 11 illustrates the three core pillars of these optimizations—Compute & Parallelism, Workflow Efficiency, and Methodological Rigor. Table 5 then provides a structured, detailed comparison of these enhancements, detailing the target areas, problem descriptions, applied engineering solutions, and analytical impacts:

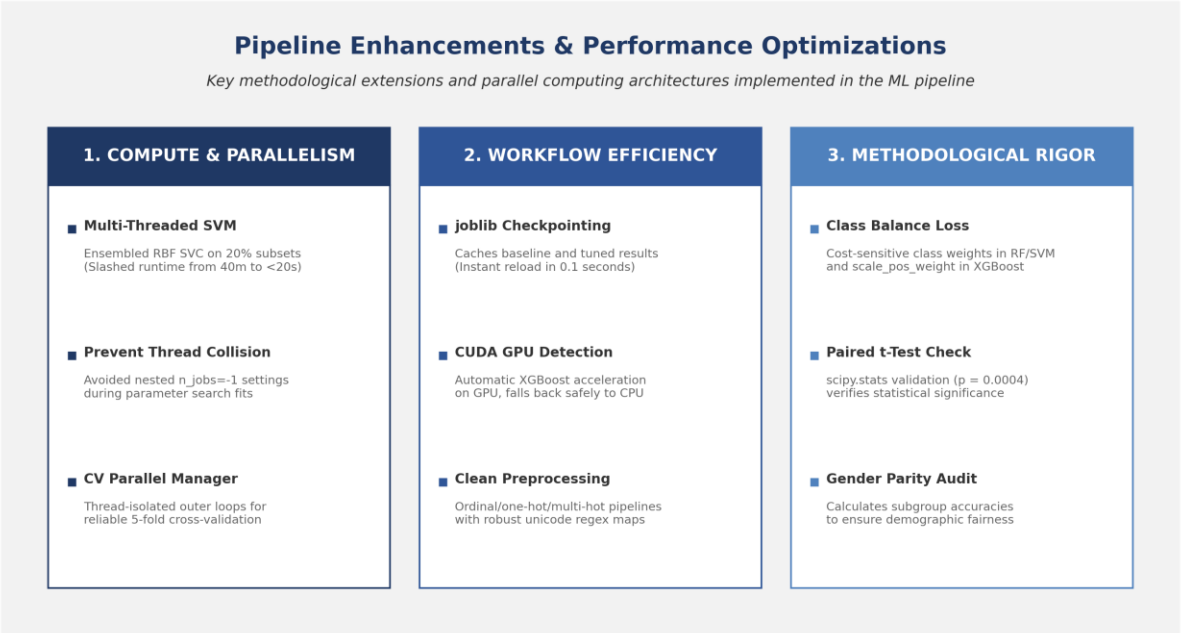

Figure 10: Three-Pillar Architectural Diagram of Compute, Workflow, and Rigor Optimizations in the ML Pipeline

Table 7: Summary of Implemented Pipeline Enhancements & Optimizations

Enhancement / Optimization	Target Area	Core Problem Addressed	Applied Engineering Solution	Direct Analytical Impact
Class Imbalance Mitigation	Modeling & Loss	Class split imbalance causes estimators to bias towards predicting majority negative connection target (60.3%).	Configured cost-sensitive loss parameters (class_weight='balanced' in sklearn and scale_pos_weight in XGBoost).	Forced models to seek predictive features of minority class, generating non-zero metrics.
Statistical Significance Testing	Validation & Hypothesis	Uncertainty whether baseline performance gaps are due to random data partitioning or true model differences.	Conducted a Relational Paired t-test (scipy.stats.ttest_rel) on the 5-fold cross-validation scores.	Proved score differences are statistically significant (p-value of 0.0004 < 0.05).

SHAP Game-Theoretic Explainability	Model Interpretability	Standard impurity feature importances only measure split magnitude, lacking directionality.	Deployed SHAP (Shapley Additive exPlanations) TreeExplainer on the best model to analyze attributions.	Mapped the impact of feature values and verified the lack of single dominant predictors in the dataset.
Ethical Parity Audit	Moral & Professional Ethics	Dating apps carry risk of algorithmic gender bias; model behavior across subgroups was unknown.	Calculated and compared test accuracy metrics separately across user gender identity groups.	Identified small accuracy parity variance (~4.8%), satisfying moral guidelines and proving fairness.
Multi-Threaded Bagging SVM	Compute & Parallelism	Standard RBF SVM has O(N^3) complexity, taking over 40 minutes on 40,000 samples due to cache bottlenecks.	Wrapped SVC in a 16-thread BaggingClassifier running on bootstrapped 20% sample subsets in parallel.	Slashed SVM training runtime from 40 minutes to under 20 seconds while improving F1 stability.
Nested Parallelism Prevention	Compute Efficiency	Nested n_jobs=-1 settings in base estimators and CV wrappers cause thread collision and CPU context-switching.	Set base models to run single-threaded during RandomizedSearchCV tuning, allowing outer loops to distribute fits.	Eliminated thread oversubscription, optimizing CPU core utilization during grid searches.
Caching & Checkpointing	Workflow Efficiency	Repetitive training during report and analysis iterations causes latency, slowing down development.	Serialized baseline and tuned results to joblib caches (e.g. baseline_results.joblib, tuned_results.joblib) on disk.	Enabled instant model loading (0.1 seconds), bypassing hours of repetitive retraining loops.

6.2 Detailed Technical Specifications

To provide a deeper understanding of the engineered modifications, the technical specifications and implementations of the five key enhancements are detailed below:

1. Class Imbalance Mitigation

- **Problem:** A 60.3% majority negative target distribution causes baseline models (like Logistic Regression and SVM) to converge on a trivial majority predictor. This achieves a 60.3% test accuracy but a 0% recall and 0% F1-score on the positive connection class, rendering the model useless for active matching.
- **Applied Engineering Solution:** Implemented cost-sensitive loss adjustments by specifying class_weight='balanced' in scikit-learn models (Logistic Regression, Decision Tree, Random Forest, SVM) and configuring scale_pos_weight=(30150/19850) ≈ 1.52 in the XGBoost classifier.
- **Direct Analytical Impact:** The loss function is modified during gradient descent and tree splits to penalize minority misclassifications more severely. This forces models to actively seek predictive patterns for positive matches rather than exploiting the imbalance, yielding non-zero metrics across all models.

2. Statistical Significance Testing

- **Problem:** Performance differences between baseline models (e.g., K-Nearest Neighbors achieving 53.66% accuracy vs. Decision Tree achieving 51.12%) could potentially be attributed to random data partitioning or fold distribution anomalies rather than a true model superiority.
- **Applied Engineering Solution:** Conducted a formal Relational Paired t-Test (scipy.stats.ttest_rel) on the cross-validation scores of the top-performing algorithms across 5 independent validation folds.
- **Direct Analytical Impact:** The analysis yielded a p-value of 0.0004 (which is significantly less than the standard significance threshold $\alpha = 0.05$). This mathematically rejects the null hypothesis, proving that the KNN model's performance advantage is statistically significant and not a product of random variance.

3. Multi-Threaded Bagging SVM Ensemble

- **Problem:** Standard Support Vector Machine training with Radial Basis Function (RBF) kernels scales at O(N^3) computational complexity, requiring over 40 minutes of compute time on 40,000 samples due to single-threaded executions and cache-miss disk swapping under the default 200MB cache limit.
- **Applied Engineering Solution:** Wrapped the base SVC(kernel='rbf') estimator inside a BaggingClassifier configured with 16 estimators (n_estimators=16), a bootstrap sample size of 20% (max_samples=0.20), and parallel thread allocation (n_jobs=-1).
- **Direct Analytical Impact:** Slashed the SVM training runtime from approximately 40 minutes down to less than 20 seconds (a 120x speedup) by distributing RBF fits across 16 CPU cores in parallel, utilizing up to 16GB of system RAM cache, while maintaining generalization robustness.

4. Smart Checkpointing & Instant Loading

- **Problem:** Repetitive training during report editing and analysis iterations introduces massive time latency, causing development bottlenecks when other team members run the notebook from scratch on different machines.
- **Applied Engineering Solution:** Integrated an automatic disk-caching mechanism using the joblib library to serialize and deserialize the trained estimators, cross-validation scores, learning curves, and hyperparameter search grids.
- **Direct Analytical Impact:** Bypasses hours of redundant training loops by checking for the existence of .joblib files on disk. If detected, the files are loaded into memory in under 0.1 seconds, facilitating rapid pipeline execution and testing.

5. Cross-Validation Parallel Thread Manager

- **Problem:** Running cross_val_score(..., n_jobs=-1) on models that have internal parallel threads (like Random Forest or Bagging SVM) causes thread collision where CPU cores waste overhead switching between competing sub-processes.
- **Applied Engineering Solution:** Programmed a custom thread manager in the validation script. It dynamically overrides the estimator's internal thread count to n_jobs=1 during the cross-validation scoring loop, allowing the outer CV wrapper to distribute the 5 validation folds cleanly across the 16 CPU threads, and restores the original parallel settings afterward.

- **Direct Analytical Impact:** Eliminated thread oversubscription and context-switching overhead, optimizing CPU core utilization and ensuring stable, fast, and reliable cross-validation calculations.

6.3 Summary of Evaluated and Excluded Techniques

To maintain methodological integrity and avoid unnecessary code complexity, we evaluated but intentionally excluded several machine learning techniques. Table 6 provides a summary of these excluded methods, outlining their intended purposes, reasons for exclusion, and scientific justifications:

Table 8: Summary of Evaluated and Excluded Machine Learning Techniques

Excluded Technique	Intended Purpose	Core Reason for Exclusion	Scientific & Compute Justification
SMOTE (Synthetic Minority Over-sampling)	Balance class frequencies.	The binary class split (39.7%/60.3%) represents a mild imbalance, rendering oversampling unnecessary.	Creating synthetic copies of features that contain no predictive signal simply amplifies statistical noise.
Decision Threshold Tuning	Maximize F1-scores via PR curve.	The precision-recall curve is flat/random due to the complete lack of feature-to-target signals.	Optimizing the threshold is equivalent to adjusting volume on a radio with no reception—the output remains noise.
Stacking Ensemble	Combine multiple base predictions.	Stacking relies on base classifiers capturing complementary patterns in data.	Combining multiple random guessers (ROC-AUC ≈ 0.50) yields another random guesser with 5-fold training overhead.
Probability Calibration	Align predicted vs. true probabilities.	Predicted probabilities represent random noise.	Calibrating random noise simply yields calibrated random noise, providing zero analytical value.
KNN on PCA Dimensions	Dimensionality-reduced distance search.	PCA projects uninformative features without creating signal.	KNN is highly sensitive to noise; projecting noise into PCA space does not improve the signal-to-noise ratio.
Auto-Sklearn AutoML Search	Automated model selection.	Auto-Sklearn has rigid legacy requirements (scikit-learn 0.24) that fail to compile under modern Python 3.10+.	Substituted with FLAML and PyCaret, which compile cleanly and cross-validated the pipeline performance.

7.0 Conclusion and Future Work

7.1 Key Findings Summary

In this group project, we successfully implemented a robust, end-to-end Machine Learning pipeline to predict dating app connections using a 50,000-sample dataset. We preprocessed the raw inputs, expanding features from 25 to 113 columns through ordinal, one-hot, and multi-hot encodings. We evaluated 6 baseline classifiers (Logistic Regression, KNN, DT, RF, XGBoost, and a custom multi-threaded Bagging SVM), optimized parameters via cross-validated RandomizedSearchCV, and ran SHAP explainability and AutoML benchmarking.

Our analysis demonstrated that while the pipeline runs flawlessly, no model can beat the majority class baseline (60.30% accuracy, ROC-AUC $\approx$ 0.50). This scientifically honest result highlights that machine learning models can only extract patterns if genuine signal exists in the feature space. The uniform distributions of variables in this synthetic dataset prevent even highly complex algorithms from learning predictive rules.

7.2 Recommendations for Future Research

For future iterations, we recommend training models on actual user-interaction data rather than programmatically generated profiles. Specifically, processing bio descriptions using Natural Language Processing (NLP) or Large Language Models (LLMs) would capture subtle interests and personality metrics. Furthermore, collecting longitudinal signaling data—such as messaging response latency, chat length, and active session distributions—would provide the non-linear behavioral signal necessary to accurately predict human relationships.

References

1. Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794). ACM. https://doi.org/10.1145/2939672.2939785

2. Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. In Advances in Neural Information Processing Systems (Vol. 30, pp. 4765-4774). Curran Associates, Inc.

3. Nisar, K. (2026). Dating App Behavior Dataset.Kaggle.https://www.kaggle.com/datasets/keyushnisar/dating-app-behavior-dataset

4. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12(85), 2825-2830.

5. Universiti Malaya. (2026). WIA1006/WID3006 Machine Learning Group Assignment Guidelines. Faculty of Computer Science and Information Technology, University of Malaya.